Project Report: Sales Intelligence Hub

1. Project Overview

Organizations operating across multiple branches often face challenges in tracking sales, managing payment collections, and maintaining structured financial records. Manual tracking methods result in data inconsistencies and incorrect calculations.

The **Sales Intelligence Hub** is a structured, branch-based Sales Management System designed to solve these issues. It provides a centralized database with automated financial calculations and an interactive web dashboard for real-time business monitoring.

2. Technologies & Tools Used

Category	Technology	Purpose
Frontend UI	Streamlit	Building the interactive, web-based dashboard.
Backend Logic	Python	Routing data, handling user sessions, and processing inputs.
Database	MySQL	Storing relational data, enforcing constraints, and automating math via Triggers.
Data Handling	Pandas	Executing SQL queries and formatting the results into clean dataframes.

3. Key Features & Advantages

- **Role-Based Access Control (RBAC):** Secure login system where Super Admins can monitor the entire business, while Branch Admins are restricted to managing only their specific locations.
- **Automated Financial Tracking:** Utilizes MySQL AFTER INSERT triggers to completely eliminate manual math. When a payment is recorded, the database automatically updates the received amount and calculates the pending balance.
- **Real-Time KPI Monitoring:** A dedicated financial insights tab displays overall revenue, pending collection percentages, and payment method breakdowns (Cash/UPI/Card).
- **Built-in Analytics Tool:** Features a custom SQL execution environment allowing administrators to run database queries directly from the web interface without needing backend database access.
- **Data Integrity:** Strict relational database design ensures that no orphan records exist and all payments are perfectly tied to verified sales and branches.

4. Database Architecture

The system is built on a normalized relational database containing four primary tables:

1. **branches**: Stores branch locations and administrator details.
2. **users**: Manages secure login credentials and role assignments.
3. **customer_sales**: The core ledger tracking gross sales, received amounts, and dynamically generated pending balances.
4. **payment_splits**: Logs individual transactions and payment methods, firing automation triggers to update the main sales ledger.

5. Complete Application Source Code

```
import streamlit as st
import mysql.connector
import pandas as pd

# Database
def get_db_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="Alpha@69",
```

```
        database="sales_management"
    )

# Login
def login_user(username, password):
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM users WHERE username=%s AND password=%s",
(username, password))
    user = cursor.fetchone()
    conn.close()
    return user

# Session
if 'logged_in' not in st.session_state:
    st.session_state['logged_in'] = False
if 'user_role' not in st.session_state:
    st.session_state['user_role'] = None
if 'branch_id' not in st.session_state:
    st.session_state['branch_id'] = None

# Login Screen
if not st.session_state['logged_in']:
    st.title("Sales Intelligence Hub - Login")
    username = st.text_input("Username")
    password = st.text_input("Password", type="password")

    if st.button("Login"):
        user = login_user(username, password)
        if user:
            st.session_state['logged_in'] = True
            st.session_state['user_role'] = user['role']
            st.session_state['branch_id'] = user['branch_id']
            st.success("Login Successful!")
            st.rerun()
        else:
            st.error("Invalid credentials. Please try again.")

# Dashboard
else:
    st.sidebar.title("Navigation")
    if st.sidebar.button("Logout"):
        st.session_state.clear()
        st.rerun()
```

```
    st.title("Sales Management Dashboard")
    st.write(f"Logged in as: **{st.session_state['user_role']}**")

    # Navigation
    menu = [
        "View Sales Report",
        "Add New Sale",
        "Add Payment Split",
        "Financial KPI Summary",
        "SQL Queries",
        "SQL Q/A"
    ]
    choice = st.sidebar.selectbox("Select Action", menu)

    # 1.View Sales Report
    if choice == "View Sales Report":
        st.subheader("Sales Report")
        conn = get_db_connection()

        if st.session_state['user_role'] == 'Super Admin':
            branches_df = pd.read_sql("SELECT branch_id, branch_name FROM branches", conn)
            branch_dict = dict(zip(branches_df.branch_name, branches_df.branch_id))

            filter_choice = st.selectbox("Filter by Branch", ["All Branches"] + list(branch_dict.keys()))

            if filter_choice == "All Branches":
                df = pd.read_sql("SELECT * FROM customer_sales ORDER BY date DESC", conn)
            else:
                b_id = branch_dict[filter_choice]
                df = pd.read_sql(f"SELECT * FROM customer_sales WHERE branch_id = {b_id}
ORDER BY date DESC", conn)
        else:
            df = pd.read_sql(f"SELECT * FROM customer_sales WHERE branch_id =
{st.session_state['branch_id']} ORDER BY date DESC", conn)

        st.dataframe(df)
        conn.close()

    # 2.Add New Sale
    elif choice == "Add New Sale":
        st.subheader("Record a New Sale")
        with st.form("new_sale_form"):
            if st.session_state['user_role'] == 'Super Admin':
```

```
            b_id = st.number_input("Branch ID", min_value=1, step=1)
        else:
            b_id = st.session_state['branch_id']
            st.write(f"Recording for Branch ID: {b_id}")

        date = st.date_input("Date")
        customer_name = st.text_input("Customer Name")
        mobile = st.text_input("Mobile Number")
        product = st.text_input("Product Name")
        gross_sales = st.number_input("Gross Sales", min_value=0.0)

        submitted = st.form_submit_button("Submit Sale")
        if submitted:
            conn = get_db_connection()
            cursor = conn.cursor()
            query = """INSERT INTO customer_sales (branch_id, date, customer_name,
mobile_number, product_name, gross_sales)
                    VALUES (%s, %s, %s, %s, %s, %s)"""
            try:
                cursor.execute(query, (b_id, date, customer_name, mobile, product, gross_sales))
                conn.commit()
                st.success("Sale recorded successfully!")
            except Exception as e:
                st.error(f"Error: {e}")
            finally:
                conn.close()

    # 3.Add Payment Split
    elif choice == "Add Payment Split":
        st.subheader("Record a Partial or Full Payment")
        with st.form("payment_form"):
            sale_id = st.number_input("Sale ID", min_value=1, step=1)
            p_date = st.date_input("Payment Date")
            amount = st.number_input("Amount Paid", min_value=1.0)
            method = st.selectbox("Payment Method", ["Cash", "UPI", "Card"])

            submitted = st.form_submit_button("Record Payment")
            if submitted:
                conn = get_db_connection()
                cursor = conn.cursor()
                query = "INSERT INTO payment_splits (sale_id, payment_date, amount_paid,
payment_method) VALUES (%s, %s, %s, %s)"
                try:
                    cursor.execute(query, (sale_id, p_date, amount, method))
```

```
                conn.commit()
                st.success("Payment recorded. Sale balance updated automatically!")
            except Exception as e:
                st.error(f"Error: {e}")
            finally:
                conn.close()

    # 4.Financial KPI Summary
    elif choice == "Financial KPI Summary":
        st.subheader("Business Metrics & Insights")
        conn = get_db_connection()

        if st.session_state['user_role'] == 'Super Admin':
            kpi_query = "SELECT SUM(gross_sales) as TS, SUM(received_amount) as TR,
SUM(pending_amount) as TP FROM customer_sales"
            method_query = "SELECT payment_method, SUM(amount_paid) as Total FROM
payment_splits GROUP BY payment_method"
            branch_query = "SELECT b.branch_name, SUM(c.gross_sales) as Total_Sales FROM
customer_sales c JOIN branches b ON c.branch_id = b.branch_id GROUP BY b.branch_name"
        else:
            b_id = st.session_state['branch_id']
            kpi_query = f"SELECT SUM(gross_sales) as TS, SUM(received_amount) as TR,
SUM(pending_amount) as TP FROM customer_sales WHERE branch_id = {b_id}"
            method_query = f"SELECT p.payment_method, SUM(p.amount_paid) as Total FROM
payment_splits p JOIN customer_sales c ON p.sale_id = c.sale_id WHERE c.branch_id = {b_id}
GROUP BY p.payment_method"
            branch_query = None

        kpi_df = pd.read_sql(kpi_query, conn)
        total_sales = float(kpi_df['TS'].iloc[0] or 0)
        total_received = float(kpi_df['TR'].iloc[0] or 0)
        total_pending = float(kpi_df['TP'].iloc[0] or 0)
        pending_percentage = (total_pending / total_sales * 100) if total_sales > 0 else 0

        st.write("### Overall Revenue Summary")
        col1, col2 = st.columns(2)
        col3, col4 = st.columns(2)

        col1.metric("Total Sales", f"₹ {total_sales:,.0f}")
        col2.metric("Total Received", f"₹ {total_received:,.0f}")
        col3.metric("Total Pending", f"₹ {total_pending:,.0f}")
        col4.metric("Pending %", f"{pending_percentage:.2f} %")

        st.write("---")
```

```
        st.write("### Payment Method Analysis")
        method_df = pd.read_sql(method_query, conn)
        if not method_df.empty:
            st.dataframe(
                method_df,
                column_config={"payment_method": "Payment Method", "Total":
st.column_config.NumberColumn("Total Collected (₹)", format="%,.0f")},
                hide_index=True
            )

        if st.session_state['user_role'] == 'Super Admin' and branch_query:
            st.write("---")
            st.write("### Branch-wise Sales Comparison")
            branch_df = pd.read_sql(branch_query, conn)
            st.dataframe(
                branch_df,
                column_config={"branch_name": "Branch Name", "Total_Sales":
st.column_config.NumberColumn("Total Sales (₹)", format="%,.0f")},
                hide_index=True
            )
        conn.close()

    # 5.SQL Queries
    elif choice == "SQL Queries":
        st.subheader("Database Analytics Tool")
        if st.session_state['user_role'] == 'Super Admin':
            with st.form("sql_executor"):
                raw_query = st.text_area("Enter any SQL Query here (SELECT, INSERT, UPDATE,
DELETE, etc.):")
                submitted = st.form_submit_button("Run Query")

                if submitted:
                    if raw_query.strip():
                        conn = get_db_connection()
                        cursor = conn.cursor()
                        try:
                            if raw_query.strip().upper().startswith("SELECT"):
                                result_df = pd.read_sql(raw_query, conn)
                                st.success("Query Executed Successfully!")
                                st.dataframe(result_df, hide_index=True)

                            else:
                                cursor.execute(raw_query)
```

```
                        conn.commit()
                        st.success(f"Command Executed Successfully! ({cursor.rowcount} rows
affected)")

                    except Exception as e:
                        st.error(f"SQL Error: {e}")
                    finally:
                        cursor.close()
                        conn.close()
                else:
                    st.warning("Please enter a query before submitting.")
        else:
            st.error("Access Denied: Only Super Admins can execute raw SQL queries.")

    # 6.SQL Q/A
    elif choice == "SQL Q/A":
        st.subheader("Project Evaluation: SQL Questions")
        st.write("Select a category and question below to view the SQL code and its real-time
database output.")

        sql_questions = {
            "Basic Queries": {
                "1. Retrieve all records from customer_sales": "SELECT * FROM customer_sales;",
                "2. Retrieve all records from branches": "SELECT * FROM branches;",
                "3. Retrieve all records from payment_splits": "SELECT * FROM payment_splits;",
                "4. Retrieve all sales belonging to Chennai branch": "SELECT c.* FROM
customer_sales c JOIN branches b ON c.branch_id = b.branch_id WHERE b.branch_name =
'Chennai';"
            },
            "Aggregation Queries": {
                "1. Calculate total gross sales across all branches": "SELECT SUM(gross_sales) AS
Total_Gross_Sales FROM customer_sales;",
                "2. Calculate total received amount across all sales": "SELECT
SUM(received_amount) AS Total_Received_Amount FROM customer_sales;",
                "3. Calculate total pending amount across all sales": "SELECT
SUM(pending_amount) AS Total_Pending_Amount FROM customer_sales;",
                "4. Count total number of sales per branch": "SELECT branch_id, COUNT(*) AS
Total_Sales FROM customer_sales GROUP BY branch_id;",
            },
            "Join-Based Queries": {
                "1. Retrieve sales details along with branch name": "SELECT c.*, b.branch_name
FROM customer_sales c JOIN branches b ON c.branch_id = b.branch_id;",
```

```
            "2. Retrieve sales details with total payment received": "SELECT c.sale_id,
c.customer_name, SUM(p.amount_paid) AS Total_Received FROM customer_sales c LEFT
JOIN payment_splits p ON c.sale_id = p.sale_id GROUP BY c.sale_id;",
            "3. Display sales along with payment method used": "SELECT c.sale_id,
c.customer_name, p.payment_method, p.amount_paid FROM customer_sales c JOIN
payment_splits p ON c.sale_id = p.sale_id;",
            "4. Retrieve sales along with branch admin name": "SELECT c.sale_id,
c.customer_name, b.branch_admin_name FROM customer_sales c JOIN branches b ON
c.branch_id = b.branch_id;"
        },
        "Financial Tracking Queries": {
            "1. Find sales where pending amount > 5000": "SELECT * FROM customer_sales
WHERE pending_amount > 5000;",
            "2. Find branch with highest total gross sales": "SELECT b.branch_name,
SUM(c.gross_sales) AS Total_Sales FROM customer_sales c JOIN branches b ON c.branch_id
= b.branch_id GROUP BY b.branch_name ORDER BY Total_Sales DESC LIMIT 1;",
            "3. Retrieve monthly sales summary": "SELECT YEAR(date) AS Year, MONTH(date)
AS Month, SUM(gross_sales) AS Monthly_Sales FROM customer_sales GROUP BY
YEAR(date), MONTH(date) ORDER BY Year, Month;",
        }
    }

    col1, col2 = st.columns(2)
    with col1:
        category = st.selectbox("Select Query Category", list(sql_questions.keys()))
    with col2:
        question = st.selectbox("Select Specific Question", list(sql_questions[category].keys()))

    selected_query = sql_questions[category][question]
    st.write("---")
    st.write("**SQL Query:**")
    st.code(selected_query, language='sql')
    st.write("**Resulting Data:**")
    conn = get_db_connection()
    try:
        ans_df = pd.read_sql(selected_query, conn)
        if ans_df.empty:
            st.info("Query executed successfully, but no data matches this condition yet.")
        else:
            st.dataframe(ans_df, hide_index=True)
    except Exception as e:
        st.error(f"Error executing query: {e}")
    finally:
     conn.close()
```